

Smart Logix

Documentación Técnica

Por Juan Cerda y Jeremías Orellana

1. Introducción	4
2. Arquitectura del Sistema	4
2.1 Microservicios principales	4
2.2 Características de la arquitectura	4
2.3 Justificación	4
3. Estructura de Repositorios	5
3.1 Backend Repository	5
3.2 Frontend Repository	5
4. Backend For Frontend (BFF)	6
4.1 Funciones del BFF	6
4.2 Responsabilidades	6
4.3 Flujo de comunicación	6
5. Comunicación entre servicios	6
5.1 Justificación de REST	7
6. Multi-Tenancy (Path-Based + Schema Isolation)	7
6.1 Resolución del tenant (Path-Based)	7
6.2 Aislamiento de datos (Schema-Based)	7
6.3 Flujo del sistema	7
6.4 Beneficios del enfoque	8
6.5 Consideraciones técnicas	8
7. Estructura General del Proyecto	8
Backend	8
Frontend	8
8. Patrones Arquitectónicos	8
8.1 API Gateway / BFF Pattern	8
Funciones:	8
8.2 Database per Service	9
Beneficios:	9
9. Patrones de Diseño	9
9.1 Repository Pattern	9
9.2 DTO Pattern	9
9.3 Factory Pattern	9
9.4 Container/Presentational (Frontend)	9
10. Arquitectura del Sistema	9
"Ecommerce Logístico Modular"	9
Características:	9
11. Estrategia de Branching	10
12. Buenas Prácticas	10
Backend	10
Frontend	10
Infraestructura	10
13. Pruebas Unitarias	10
Backend	10
Frontend	10

14. Docker y Despliegue	11
15. Conclusión	11

1. Introducción

SmartLogix es una plataforma web de ecommerce orientada a pequeñas y medianas empresas (PYMEs), enfocada en la gestión de inventario, pedidos y logística.

El objetivo del sistema es centralizar procesos comerciales y operativos mediante una arquitectura modular basada en microservicios, permitiendo escalabilidad progresiva sin requerir infraestructura compleja.

El sistema está diseñado con un frontend desacoplado y un backend distribuido compuesto por un Backend For Frontend (BFF) y microservicios independientes.

2. Arquitectura del Sistema

SmartLogix utiliza una arquitectura híbrida basada en:

- Backend For Frontend (BFF)
 - Microservicios independientes
 - Comunicación REST síncrona
 - Base de datos por servicio
 - Multi-tenancy basado en path routing
-

2.1 Microservicios principales

- Gestión de usuarios (BFF / futuro servicio dedicado)
 - Gestión de inventario (inventory-service)
 - Gestión de pedidos (futuro)
 - Gestión logística (futuro)
-

2.2 Características de la arquitectura

- Separación por dominios
 - Despliegue independiente por servicio
 - Comunicación mediante APIs REST
 - Aislamiento de responsabilidades
 - Escalabilidad progresiva
-

2.3 Justificación

La arquitectura fue seleccionada por su equilibrio entre simplicidad y escalabilidad, permitiendo iniciar con una base liviana pero preparada para crecimiento progresivo del sistema.

3. Estructura de Repositorios

El sistema está dividido en dos repositorios principales:

3.1 Backend Repository

Contiene todos los servicios backend:

- Backend For Frontend (Spring Boot)
- inventory-service (Spring Boot)
- futuros microservicios

Estructura:

Backend/

```
|— BackendForFrontend/
|   |— auth/
|   |— product/
|   |— config/
|   |— shared (solo utilidades internas si aplica)
|
|— inventory-service/
|   |— product/
|   |— config/
|   |— repositories/
```

3.2 Frontend Repository

Aplicación frontend desarrollada en Svelte + Vite:

SmartLogix-Front/

```
|— src/
|   |— lib/
|   |   |— components/
|   |   |— services/
|   |   |— types/
```

| └─ routes/
| └─ assets/

4. Backend For Frontend (BFF)

El sistema implementa un Backend For Frontend que actúa como capa intermedia entre el frontend y los microservicios.

4.1 Funciones del BFF

- Agregación de datos desde microservicios
 - Manejo de autenticación JWT
 - Coordinación de llamadas REST
 - Adaptación de respuestas para el frontend
 - Inyección de contexto de tenant
-

4.2 Responsabilidades

- Exponer APIs simplificadas al frontend
 - Aplicar reglas de negocio de composición
 - Centralizar seguridad a nivel de entrada
-

4.3 Flujo de comunicación

Frontend (Svelte)

↓

Backend For Frontend

↓

Inventory Service

5. Comunicación entre servicios

SmartLogix utiliza comunicación síncrona basada en REST.

5.1 Justificación de REST

- Simplicidad de implementación
 - Fácil depuración
 - Bajo costo operativo
 - Adecuado para MVP y PYMEs
 - Compatibilidad con infraestructura Docker
-

6. Multi-Tenancy (Path-Based + Schema Isolation)

SmartLogix implementa un modelo híbrido de multi-tenancy.

6.1 Resolución del tenant (Path-Based)

El tenant se identifica a través de la URL utilizando el Backend For Frontend (BFF).

Ejemplo:

smartlogix.com/empresaA/products

El BFF extrae el identificador del tenant desde el path.

6.2 Aislamiento de datos (Schema-Based)

Cada tenant posee un schema independiente dentro de la base de datos.

Ejemplo:

- schema empresaA
- schema empresaB

Cada schema contiene su propio conjunto de tablas.

6.3 Flujo del sistema

Frontend (/empresaA/products)

→ BFF (resuelve tenant)

→ Backend (selecciona schema empresaA)

→ Base de datos (ejecución en schema correspondiente)

6.4 Beneficios del enfoque

- Aislamiento fuerte a nivel de persistencia
 - Evita filtrado manual por tenant_id
 - Reduce riesgo de fuga de datos
 - Mantiene simplicidad en la capa de routing
-

6.5 Consideraciones técnicas

- Requiere resolución dinámica de schema en backend
 - Migraciones deben aplicarse por schema
 - Incrementa complejidad en la capa de persistencia
-

7. Estructura General del Proyecto

Backend

Backend/

|— BackendForFrontend/

|— inventory-service/

Frontend

SmartLogix-Front/

8. Patrones Arquitectónicos

8.1 API Gateway / BFF Pattern

El BFF actúa como punto único de entrada para el frontend.

Funciones:

- Routing de requests
 - Seguridad centralizada
 - Simplificación de APIs
-

8.2 Database per Service

Cada microservicio posee su propia base de datos independiente.

Beneficios:

- Bajo acoplamiento
 - Escalabilidad modular
 - Independencia de servicios
-

9. Patrones de Diseño

9.1 Repository Pattern

Implementado en Spring Boot para abstraer acceso a datos.

9.2 DTO Pattern

Se utilizan DTOs para transferencia de datos entre capas y servicios.

9.3 Factory Pattern

Utilizado para creación de tipos de envío.

9.4 Container/Presentational (Frontend)

Separación entre lógica y UI en Svelte.

10. Arquitectura del Sistema

SmartLogix sigue un modelo:

“Ecommerce Logístico Modular”

Características:

- Arquitectura híbrida
 - Microservicios progresivos
 - BFF como capa de composición
 - Multi-tenancy basado en rutas
-

11. Estrategia de Branching

GitFlow simplificado:

- main → producción
 - develop → integración
 - feature/* → desarrollo
 - hotfix/* → correcciones
-

12. Buenas Prácticas

Backend

- separación por dominio
- DTOs por capa
- validación centralizada

Frontend

- componentes reutilizables
- separación lógica/UI

Infraestructura

- Dockerización
 - variables de entorno
-

13. Pruebas Unitarias

Backend

- JUnit
- Mockito

Frontend

- Vitest
-

14. Docker y Despliegue

El sistema utiliza Docker para:

- consistencia de entornos
 - despliegue simplificado
 - portabilidad
-

15. Conclusión

SmartLogix implementa una arquitectura híbrida basada en BFF y microservicios progresivos, con comunicación REST y multi-tenancy basado en rutas.

La eliminación del módulo compartido (**shared**) permite una mayor independencia entre servicios, reduciendo el acoplamiento y facilitando la evolución futura del sistema hacia una arquitectura de microservicios más pura.